

Static Methods of `java.util.Collections`

◆ Basic Operations

Method	Description
<code>sort(List)</code>	Sorts a list in ascending order.
<code>reverse(List)</code>	Reverses the order of elements.
<code>shuffle(List)</code>	Randomly shuffles the elements.
<code>swap(List, i, j)</code>	Swaps elements at two positions.

◆ Searching & Finding Values

Method	Description
<code>max(Collection)</code>	Returns the maximum element.
<code>min(Collection)</code>	Returns the minimum element.
<code>binarySearch(List, key)</code>	Searches in a sorted list and returns index.
<code>frequency(Collection, Object)</code>	Counts occurrences of an element.

◆ Copying & Replacing

Method	Description
<code>copy(dest, src)</code>	Copies elements from <code>src</code> → <code>dest</code> (<code>dest</code> must be same size or bigger).
<code>fill(List, value)</code>	Replaces all elements with a given value.

◆ Thread Safety & Unmodifiable Collections

Method	Description
<code>synchronizedList(List)</code>	Returns thread-safe list.
<code>synchronizedSet(Set)</code>	Returns thread-safe set.
<code>synchronizedMap(Map)</code>	Returns thread-safe map.
<code>unmodifiableList(List)</code>	Returns read-only list.
<code>unmodifiableSet(Set)</code>	Returns read-only set.
<code>unmodifiableMap(Map)</code>	Returns read-only map.

◆ Misc Utility Methods

Method	Description
<code>disjoint(Collection a, Collection b)</code>	Returns true if two collections have no common elements.
<code>emptyList()</code>	Returns an immutable empty list.
<code>emptySet()</code>	Returns an immutable empty set.
<code>emptyMap()</code>	Returns an immutable empty map.

Feature / Concept	<code>this()</code> Call Statement	<code>super()</code> Call Statement
Purpose	Calls another constructor in the same class	Calls constructor of the parent class
Relation	Same class constructor chaining	Inheritance: child → parent
Position	Must be the first line in a constructor	Must be the first line in a constructor

Feature / Concept	this() Call Statement	super() Call Statement
Default Behavior	Not inserted by Java automatically	Java inserts super() automatically if not written
Use Case	Reduce code duplication within class	Initialize parent properties before child
Example	this(10);	super(10);
Limitation	Only one this() allowed in a constructor	Only one super() allowed in a constructor
Combination	Cannot be used with super() in same constructor	Cannot be used with this() in same constructor

Feature / Usage	this Keyword	super Keyword
Refers to	Current class object	Parent class object
Access variables	Access current class variables	Access parent class variables
Access methods	this.method()	super.method()
Constructor call	this() calls current class constructor	super() calls parent class constructor
Use case	Resolve current class ambiguity	Access overridden methods or hidden variables
Implicit call	No implicit use	super() is implicitly called if not written
Inheritance relation	Only within same class	Requires inheritance
Example use	this.name = name;	super.display();